Deep Learning

Lecture 5

Recap + Agenda

- Practical Machine Learning
  - Regularization
  - Data Normalization
  - Vanishing and exploding gradients
- Programming basic ML
- Optimization Algorithms
  - Mini-batch Gradient Descent
  - Exponential Weighted Average
  - Gradient Descent with momentum
  - RMSprop
  - Adam

Optimization Algorithm

Optimization Algorithms

- Optimization algorithms helps us to ***minimize (or maximize)*** an **Objective** function (*another name for* ***Error*** *function*) **E(x)** which is simply a mathematical function dependent on the Model's internal **learnable parameters** which are used in computing the target values(**Y**) from the set of *predictors*(**X**) used in the model.
- For example — we call the **Weights(W)** and the **Bias(b)** values of the neural network as its internal learnable *parameters* which are used in computing the output values and are learned and updated in the direction of optimal solution i.e minimizing the **Loss** by the network's training process and also play a major role in the ***training*** process of the Neural Network Model .

Optimization Algorithms

- Gradient Descent
- Gradient Descent with momentum
- RMSprop
- Adam Optimization algorithm

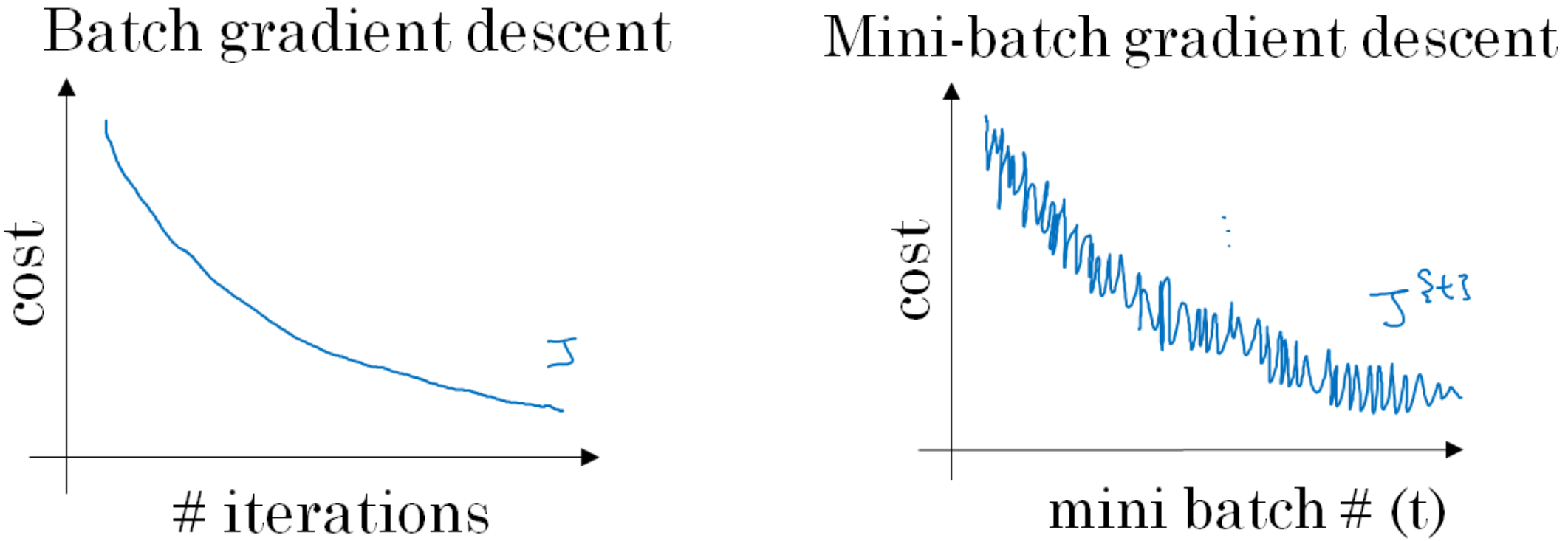
Gradient Descent

- Batch gradient descent
- mini-batch gradient descent
- Stochastic gradient descent

Mini-batch gradient descent

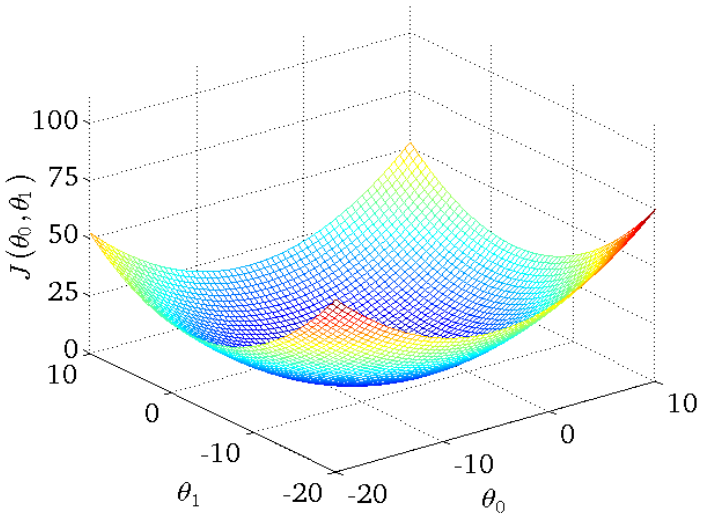
- Usually applied to training sets with large number of examples
- Take a small batch of training examples
- Compute loss on the current batch
- Compute gradients based on the loss of current batch
- Update learnable parameters
- 1 pass through all training examples is called one epoch

Training with mini-batch gradient descent

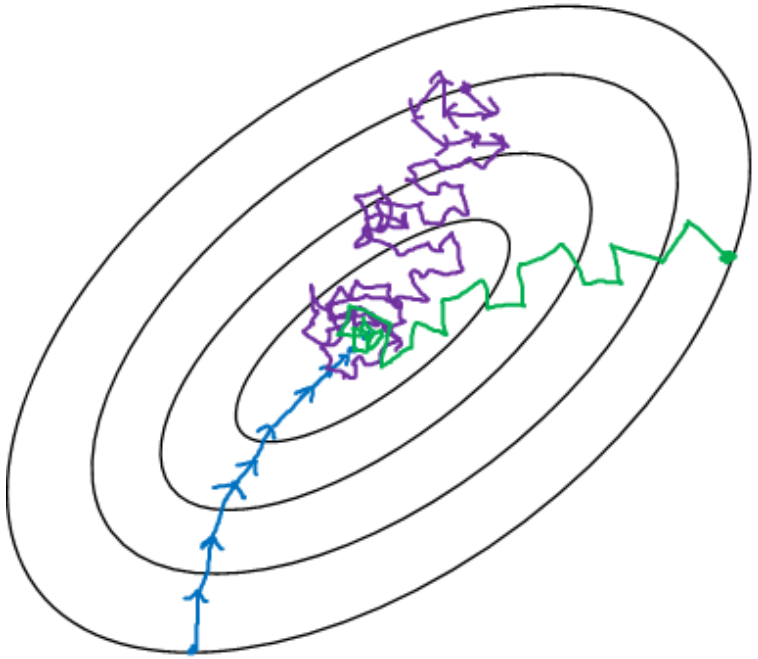


How to choose mini-batch size?

- Batch gradient descent: mini-batch size = m
- Stochastic gradient descent: mini-batch size = 1
- Mini-batch gradient descent: 1< mini-batch size < m



- If less training examples (e.g., <= 2000),
  use Batch GD
- Typical Batch-sizes: 64, 128,…, 512
- Make sure it fits in CPU/GPU memory



Exponentially Weighted Average

- A fast and efficient way to compute moving averages - implemented in the different optimization algorithms

Example: Temperature θ_t over days, calculate the moving averages

v_t: Moving average value at day 't'

$v_0 = 0$

$v_1 = 0.9v_0 + 0.1\theta_1$

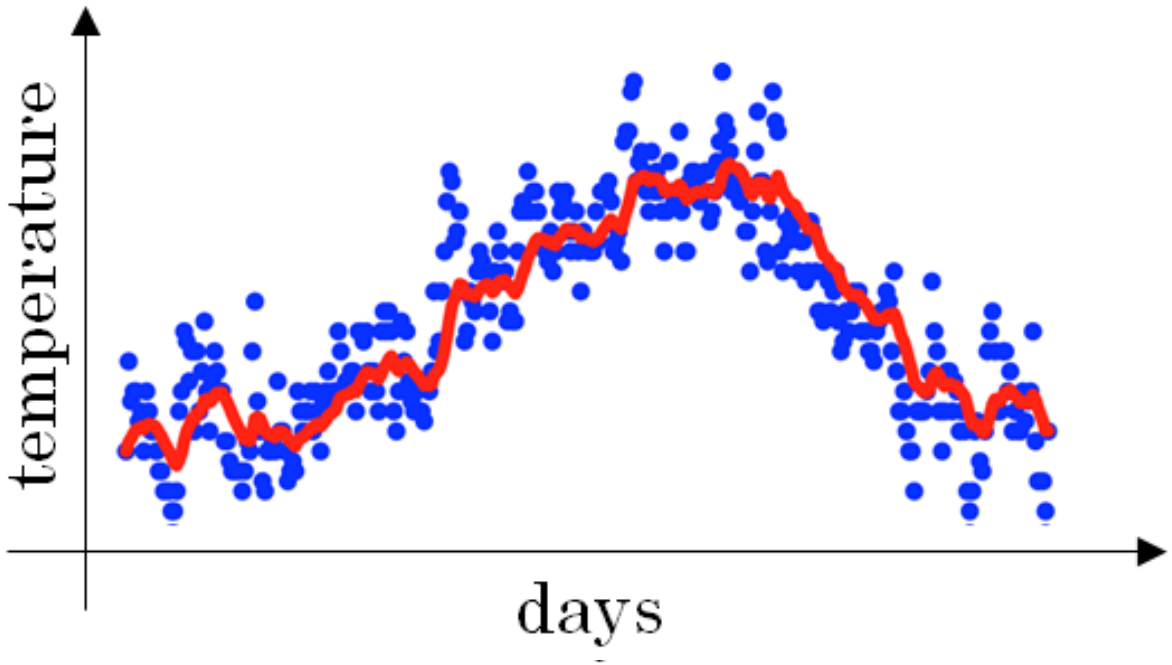
$v_1 = 0.9v_1 + 0.1\theta_2$

..

$v_t = 0.9v_{t-1} + 0.1\theta_t$

if $\beta = 0.9$

$v_t = \beta v_{t-1} + (1-\beta)\theta_t$



Exponentially Weighted Average

What does v_t and β means

v_t: averaging over $\frac{1}{1-\beta}$ days (approx)

For ex. , For β=0.9, $\frac{1}{1-\beta}$ ~=10 ;

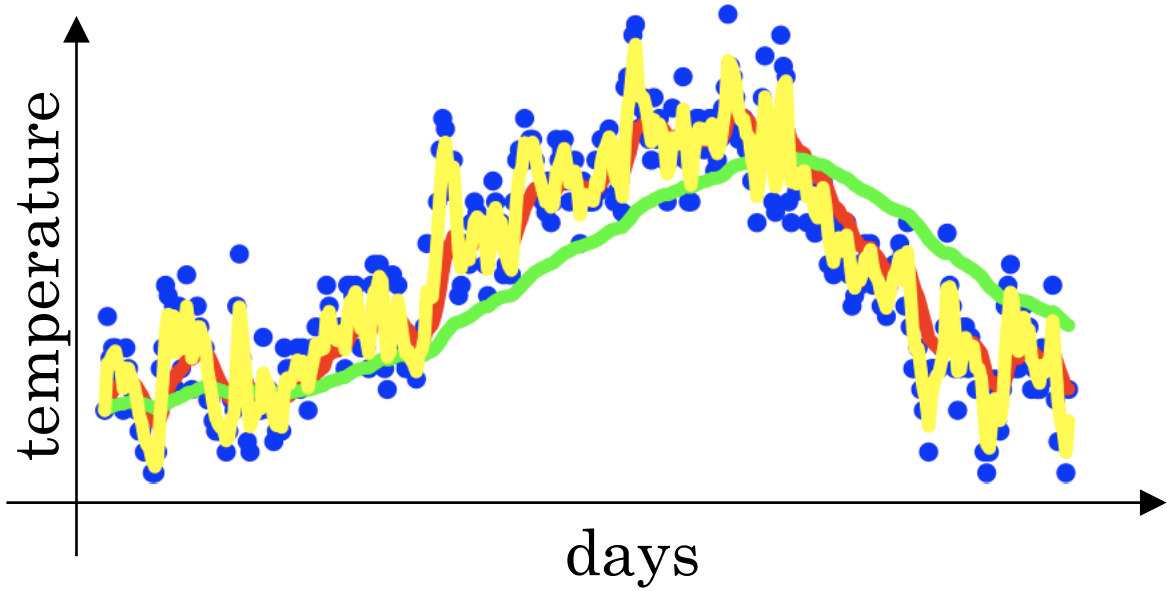
β = 0.9 averages over 10 days (smooth curve: Red Line)

For β=0.98, $\frac{1}{1-\beta}$ ~=50 ;

β = 0.98 averages over 50 days (smoother curve: Green Line)

For β=0.5, $\frac{1}{1-\beta}$ ~=2 ;

β = 0.5 averages over 2 days (Fluctuations: Yellow Line) - Much more noisy



What is Exponentially weighted average actually doing?

$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

Going backwards from v_{100},
$v_{100} = 0.1\theta_{100} + 0.9v_{99}$
$v_{99} = 0.1\theta_{99} + 0.9v_{98}$
Substituting v_{99},
$v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98})$
or, $v_{100} = 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97}))$

Generalizing,
$$v_{100} = 0.1\theta_{100} + 0.1 * 0.9 * \theta_{99} + 0.1 * (0.9)^2\theta_{98} + 0.1 * (0.9)^3\theta_{97} + +0.1 * (0.9)^4\theta_{96} + \ldots..$$

v_{100} is basically an element wise computation of two metrices/functions - one an exponential decay function containing diminishing values (0.9, 0.9^2, 0.9^3,..) and another with all the elements of θ_t.

What is Exponentially weighted average actually doing?

Also, if β = 0.9, the weight decays to about a third by 10th iteration.

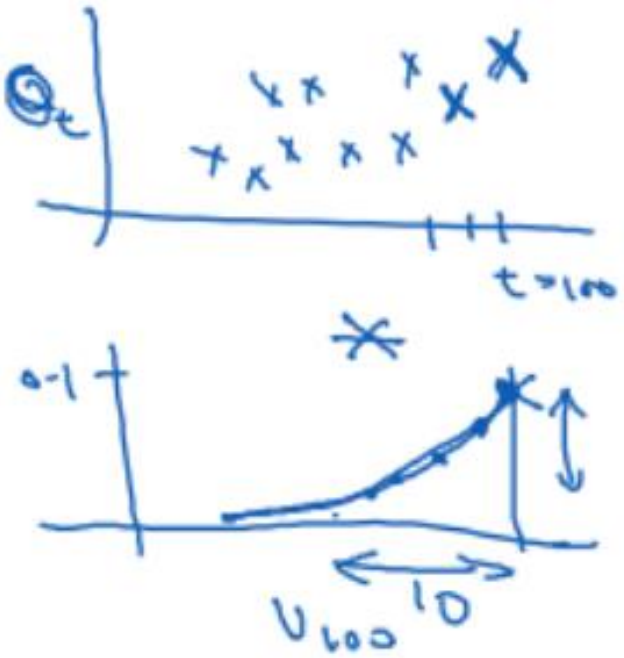
Proof: $(1-\epsilon)^{\frac{1}{\epsilon}} = \frac{1}{e}$

ϵ = 1-β

of β = 0.9, ϵ = 1-β = 0.1

$$(1-\epsilon)^{\frac{1}{\epsilon}} = 0.9^{10} = 0.35 = \frac{1}{e}$$

Interpretation: It takes about 10 days for height to decay to 1/3rd



If β = 0.98, ϵ = 0.02, $\frac{1}{\epsilon}$ = 50;

It takes approx 50 days for height to decay to 1/3rd

Implementing Exponentially Weighted Average

v_θ: v is computing exponentially weighted average of parameter θ.

day 0: $v_\theta = 0$ day 1: $v_\theta = \beta v + (1-\beta)\theta_1$
day 2: $v_\theta = \beta v + (1-\beta)\theta_2$
...

Algorithms: $v_\theta = 0$ Repeat: {

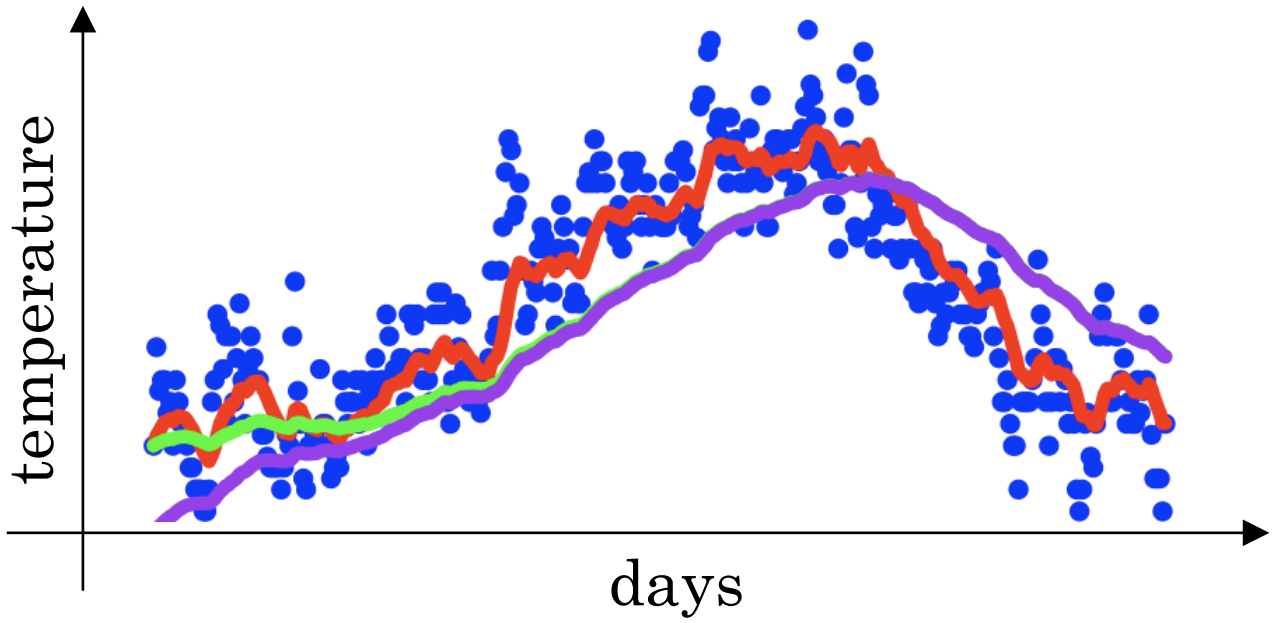
Get next θ_t

$v_\theta := \beta v_\theta + (1-\beta)\theta_t$

}

Single line implementation for fast and efficient calculation of exponentially weighted moving average.

Bias correction in Exponentially Weighted Moving Average



$$v_t = \beta v_{t-1} + (1-\beta)\theta_t$$

Figure: The ideal curve should be the GREEN one, but it starts as the PURPLE curve since the values initially are zero

Bias correction in Exponentially Weighted Moving Average

Example: Starting from t=0 and moving forward,

$v_0 = 0$ $v_1 = 0.98v_0 + 0.02\theta_1 = 0.02\theta_1$

$v_2 = 0.98v_1 + 0.02\theta_2$ = $0.0196\theta_1 + 0.02\theta_2$

The initial values of v_t will be very low which need to be compensated.

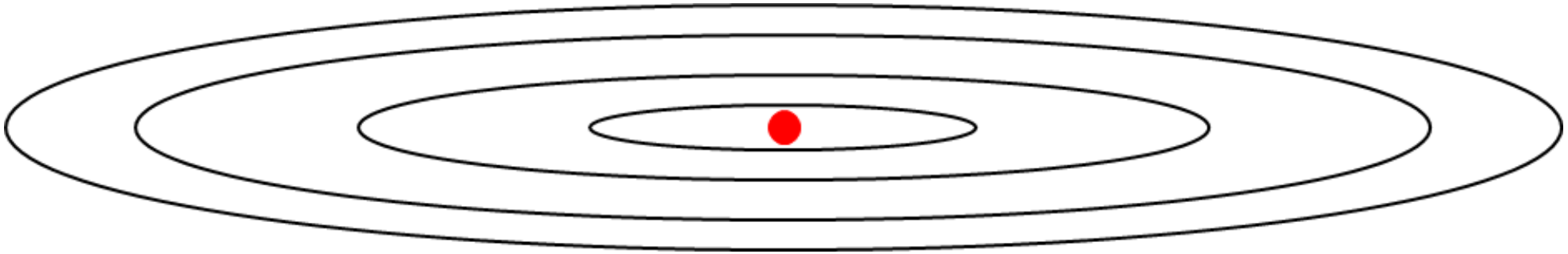
Make $v_t = \frac{v_t}{1-\beta^t}$

for t=2, $1 - \beta^t$ = $1 - 0.98^2$ = 0.0396 (Bias Correction Factor)

$$v_2 = \frac{v_2}{0.0396} = \frac{0.0196\theta_1 + 0.02\theta_2}{0.0396}$$

When t is large, $\frac{1}{1-\beta^t} = 1$, hence bias correction factor has no effect when t is sufficiently large. It only jacks up the intial values.

Momentum



On iteration t:

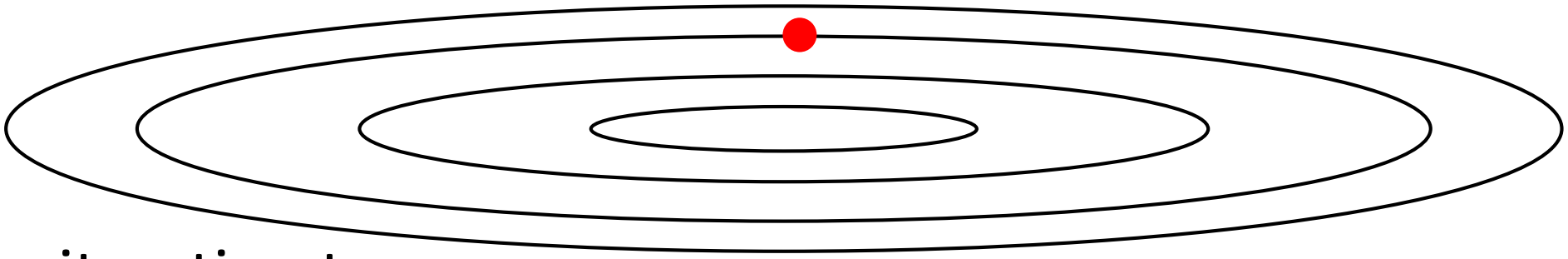
Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

RMSprop: Root Mean Square Prop



On iteration t

Compute dW and db on the mini batch

$$s_{dW} = \beta_2 s_{dW} + (1 - \beta_2) dW^2$$

$$s_{db} = \beta_2 s_{db} + (1 - \beta_2) db^2$$

$$W = W - \alpha \frac{dW}{\sqrt{s_{dW} + \epsilon}}$$

$$b = b - \alpha \frac{db}{\sqrt{s_{db} + \epsilon}}$$

Adam optimization algorithm

- Hybrid of Momentum and SMSProp

$s_{dW} = 0, v_{dW} = 0, s_{db} = 0, v_{db} = 0$

On iteration t

Compute dW and db on the mini batch

$s_{dW} = \beta_2 s_{dW} + (1 - \beta_2) dW^2, v_{dW} = \beta_1 v_{dW} + (1 - \beta_1) dW$

$s_{db} = \beta_2 s_{db} + (1 - \beta_2) db^2, v_{db} = \beta_1 v_{db} + (1 - \beta_1) db$

$$s_{dW}^{corrr} = \frac{s_{dW}}{1 - \beta_2^t}, s_{db}^{corrr} = \frac{s_{db}}{1 - \beta_2^t}$$

$$v_{dW}^{corrr} = \frac{v_{dW}}{1 - \beta_1^t}, v_{db}^{corrr} = \frac{v_{db}}{1 - \beta_1^t}$$

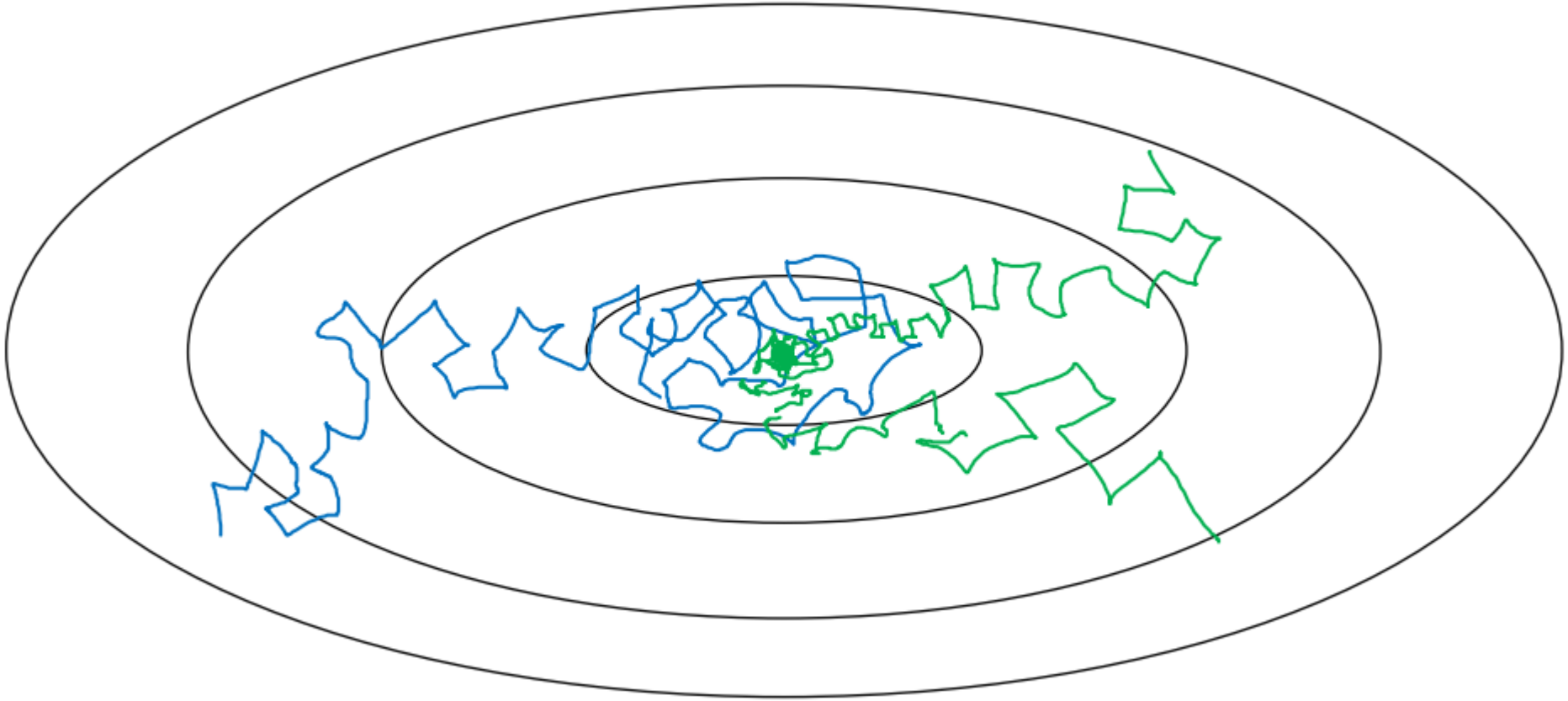
$$W = W - \alpha \frac{v_{dW}^{corrr}}{\sqrt{s_{dW}^{corrr} + \epsilon}}, b = b - \alpha \frac{v_{db}^{corrr}}{\sqrt{s_{db}^{corrr} + \epsilon}}$$

Hyper parameter choice

- Adam (Adaptive moment estimation)
- α needs to be tuned
- β_1 is by defaut 0.9
- β_2 is by defaut 0.999
- ε is by default 10^{-8}

Learning rate decay

- Slowly reduce learning rate: $\alpha=\alpha*1/(1+ \text{decay-rate}*\text{epoch-num})$



Other learning rate decay methods

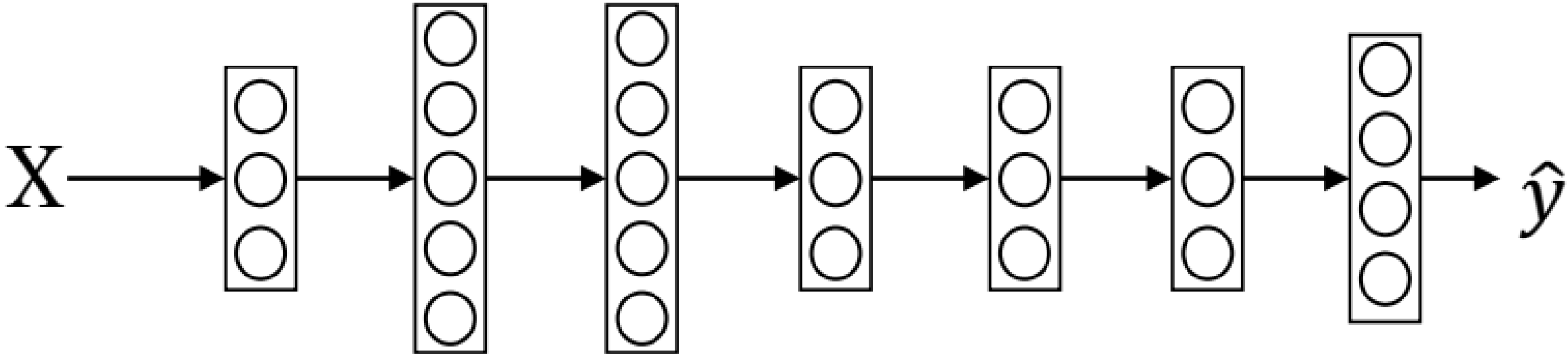
- Exponential decay
- Discrete stair case
- Manual decay

Multi-class Classification

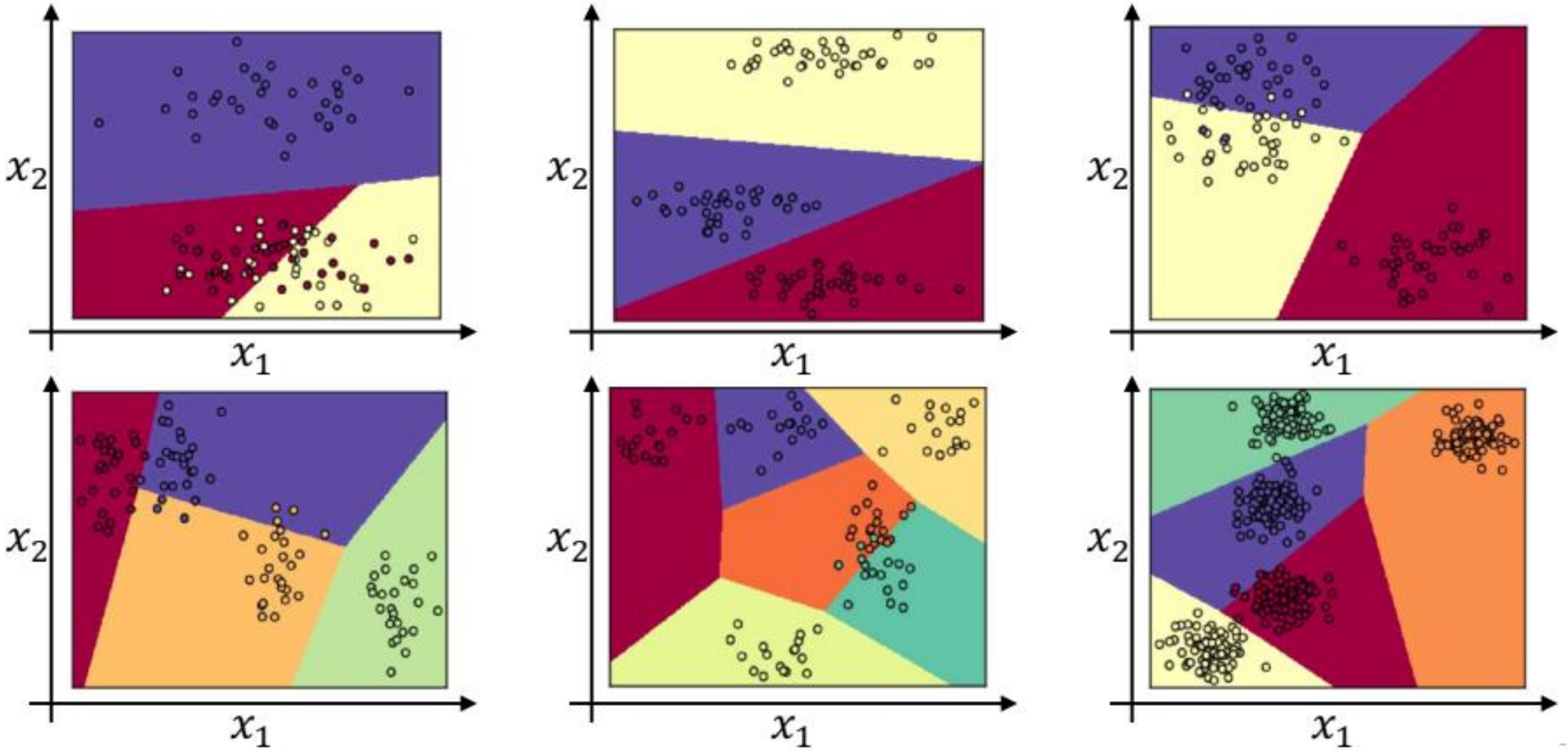
Softmax regression

Recognizing Multiple Classes

3 1 2 0 3 2 0 1



Softmax Examples



Loss Function

- $J = -\sum_{i=1}^{k} y_i log(\hat{y}_i)$
- Assuming y is one hot encoding of the class label
- Use softmax as activation function
  - Transform output into probabilities

Human-Readable → Machine-Readable

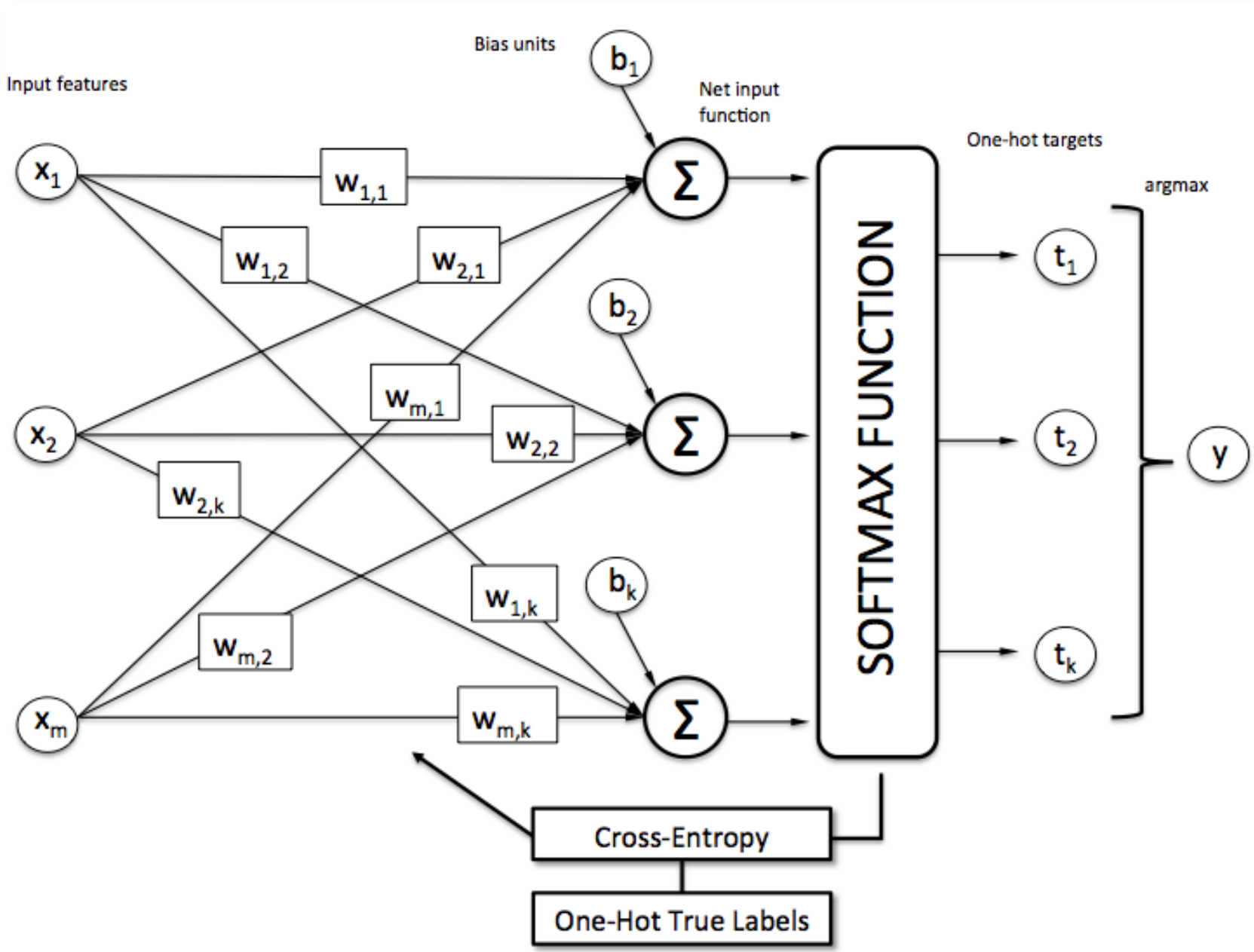
Pet
Cat
Dog
Turtle
Fish
Cat

Cat	Dog	Turtle	Fish
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

Softmax function

$$P(y=j \mid \Theta^{(i)}) = \frac{e^{\Theta^{(i)}}}{\sum_{j=0}^{k} e^{\Theta_k^{(i)}}}$$

where $\Theta = w_0 x_0 + w_1 x_1 + \ldots + w_k x_k = \sum_{i=0}^{k} w_i x_i = w^T x$



Softmax Regression